

This shows that the `/` filesystem of the guest is on `/dev/VolGroup/lv_root` and `/boot` is on `/dev/sda1`.

Next, launch an interactive guestfish filesystem shell on the image by mounting both `/` and `/boot` filesystems, appropriately.

```
# guestfish -a f12.img -n /dev/mapper/VolGroup-lv_root -n /dev/sda1:boot
```

```
Welcome to guestfish, the libguestfs filesystem interactive shell for
editing virtual machine filesystems.
```

```
Type: 'help' for help with commands
      'quit' to quit the shell
```

```
><fs> df
Filesystem            2K-blocks    Used Available Use% Mounted on
/dev/mapper/VolGroup-lv_root
6821604      1346828      5128256    21% /sysroot
/dev/sda1        198337       21435      166662    12% /sysroot/boot

><fs> grub-install /dev/sda
```

In the above command, the image or the block device on which the guest is installed should follow the `-a` switch. Each block device that needs to be mounted from the guest should follow the `-n` switch in the format of the 'block device:mount point'. We need to mount `/boot` since the `grub-install` command installs stage files in the `/boot/grub/` directory.

Template

Most virtualisation technologies support deploying virtual desktops from templates using the copy-on-write (COW) method. libguestfs will be very useful in making modifications on the template, running security updates on the template, etc.

For example, let's assume 200 virtual desktops are deployed from a template using the copy-on-write method. While using this method, changes specific to a virtual desktop from the actual template will be stored on a different file. The template will be used in conjunction with the changes on the file to form the guest filesystem. If the kernel needs to be updated on these 200 virtual desktops, it's as simple as taking a copy of the template, using libguestfs to update the kernel on the copy of the template, and changing the template for the guest to use to the new template with the updated kernel. On the next reboot, all the 200 virtual desktops should have the updated kernel. Likewise, making batch changes to configuration files is also very easy, using this method.

Windows support

libguestfs has minimal support when it comes to handling Windows-based images. FAT and NTFS filesystems are fully supported. You can use `virt-win-reg` to view the values from the Windows registry. For example, I used

the following command to get the product name from a Windows 2003 image:

```
# virt-win-reg windows-2003.img 'HKEY_LOCAL_MACHINE\Software\Microsoft\
Windows NT\CurrentVersion' ProductName
Microsoft Windows Server 2003
```

Partial back-up

Taking a full back-up of a virtual machine is very easy. Just backing up the whole image or taking lvm snapshots or 'dd-ing' block devices will suffice. On physical systems, taking a partial back-up (e.g., taking a back-up of Apache configuration files only) usually involves copying content from inside the virtual machine over the network, using `rsync`, `scp` or other back-up utilities. Libguestfs can be used to take a partial back-up of the guest from the host without logging into the guest. However, it needs to be noted that, for a reliable back-up, I strongly recommend that you do not use libguestfs. A back-up agent should be installed inside the guest, like we do with ordinary machines.

Cloning a virtual machine

Cloning a virtual machine is usually done by copying the image of a virtual machine to a different location and replacing details in the image that are unique to a virtual machine, like the hostname, IP address, MAC address, etc. You can use libguestfs to serve this purpose, whereas these unique details will either be replaced with actual details provided by the user or replaced by a placeholder.

Areas for improvement in libguestfs

One thing that I expected to see in libguestfs was support for Solaris filesystems like UFS, ZFS, etc. Unfortunately, it's not available as of now. (Note, that this isn't a libguestfs drawback, *per se*. If Linux has support for them, libguestfs will support them. Also, there are some licensing restrictions when it comes to supporting ZFS.)

Caution

One thing that needs to be kept in mind while dealing with libguestfs is not to make modifications when using it on running guests. There is a strong possibility that this will corrupt the guest filesystem beyond recovery. Future versions of libguestfs will introduce thorough locking to prevent people from doing this. At this moment, it's recommended to always connect to running guests using the `—ro` switch to guestfish, to prevent modifications. 

Resources

- libguestfs home: www.libguestfs.org
- Richard W M Jones' Blog: nmj.wordpress.com

By: Sadique Puthen

The author is an open source enthusiast with deep interest in Linux and virtualisation technologies. He is currently working as a senior technical support engineer at Red Hat, and can be reached at sputhenp@redhat.com